

05 - Clustering

Alessandro Bregoli

2025

Introduction

In this tutorial we will discuss the following topics:

-

-



Introduction

In this tutorial we will discuss the following topics:

- **Clustering Classification:**
 - Partitional/Hierarcical
 - Exclusive/Overlapping/Fuzzy
 - Complete/Partial
-



Introduction

In this tutorial we will discuss the following topics:

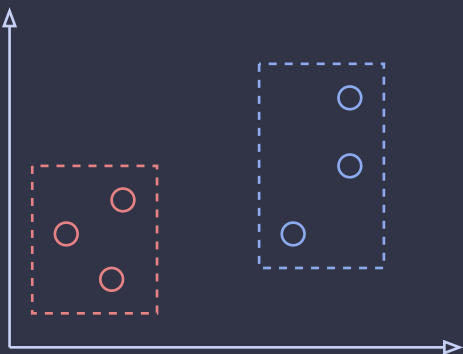
- **Clustering Classification:**
 - Partitional/Hierarcical
 - Exclusive/Overlapping/Fuzzy
 - Complete/Partial
- **Proximity Measures**
 - Euclid Distance
 - Cosine Similarity
 - Mahalanobis Distance
 - Jaccard Similarity



Clustering

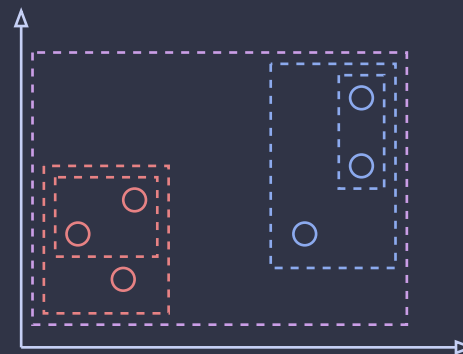
Partitional - Hierarchical

Partitional



A **partitional clustering** split the data into **non-overlapping clusters**.

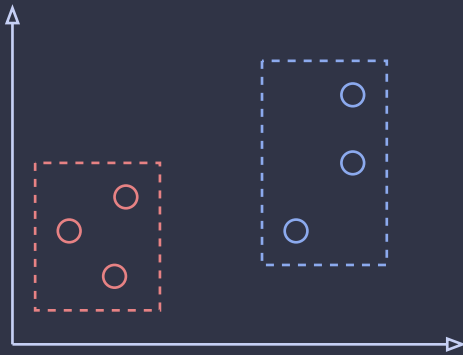
Hierarchical



A **hierarchical clustering** split the data into overlapping cluster organized as a tree.

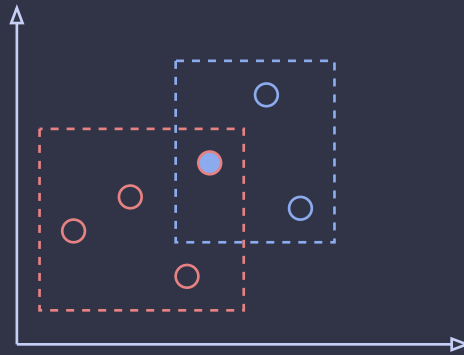
Exclusive - Overlapping - Fuzzy

Exclusive



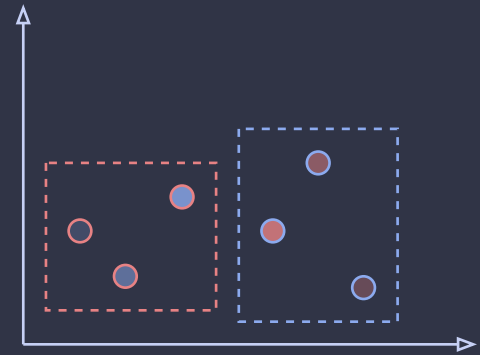
Each **object** is assigned to a **single cluster**.

Overlapping



Each **object** can be assigned to **more than one cluster**.

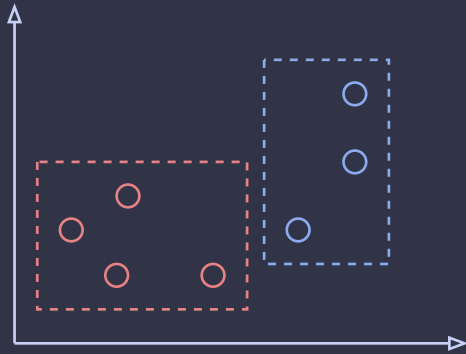
Fuzzy



Each **object** belongs to **each cluster** with a membership **weight**.

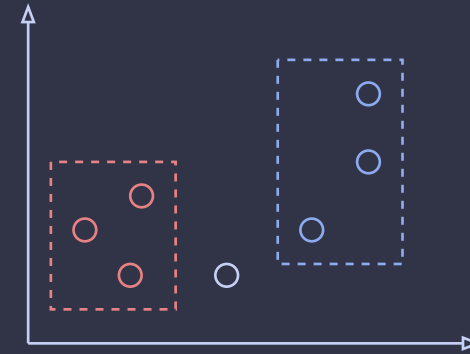
Complete - Partial

Complete



All the **objects** are **assigned** to at least one **cluster**.

Partial



An **object** can be assigned to **no cluster**.

K-means node

The *k-Means* node is provided by the standard Knime library.

k-Means



K-means node

The *k-Means* node is provided by the standard Knime library.

The k-means can be categorized as follows:

- **Partitional**: split the dataset into partitions.
- **Exclusive**: each element belong at most to one cluster.
- **Complete**: each element is assigned to a cluster.

k-Means



K-means node

The *k-Means* node is provided by the standard Knime library.

The k-means can be categorized as follows:

- **Partitional**: split the dataset into partitions.
- **Exclusive**: each element belong at most to one cluster.
- **Complete**: each element is assigned to a cluster.

The main **parameter** of the **k-means** algorithm is the **number of clusters**.

k-Means



K-means node

The *k-Means* node is provided by the standard Knime library.

The k-means can be categorized as follows:

- **Partitional**: split the dataset into partitions.
- **Exclusive**: each element belong at most to one cluster.
- **Complete**: each element is assigned to a cluster.

The main **parameter** of the **k-means** algorithm is the **number of clusters**.

The **k-means** algorithm implemented by Knime is based on **Euclid distance**. Thus, it is fundamental to **normalize** the dataset before performing the clustering.

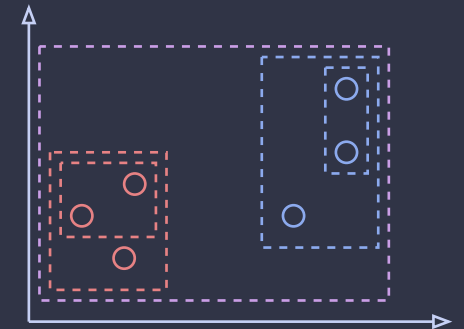
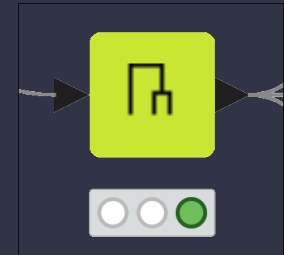
k-Means



Hierarchical Clustering Node

The *Hierarchical Clustering* node is provided by the standard Knime library.

Hierarchical Clustering



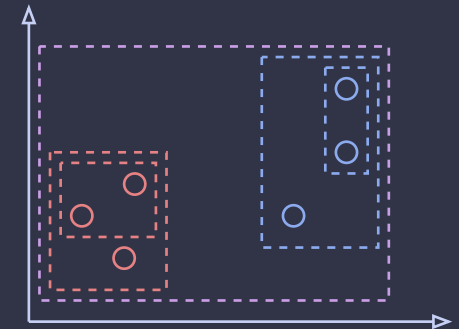
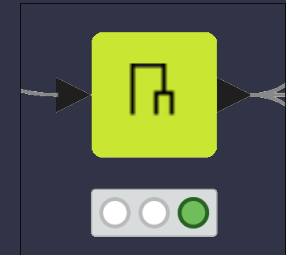
Hierarchical Clustering Node

The *Hierarchical Clustering* node is provided by the standard Knime library.

The hierarchical clustering can be categorized as follows:

- **Hierarchical:** split the dataset into overlapping cluster organized as a tree.
- **Exclusive/Overlapping:** each node can be assigned to more than one cluster. However, the knime implementation of this algorithm extract only exclusive clusters.
- **Complete:** each element is assigned to a cluster.

Hierarchical Clustering



Hierarchical Clustering Node

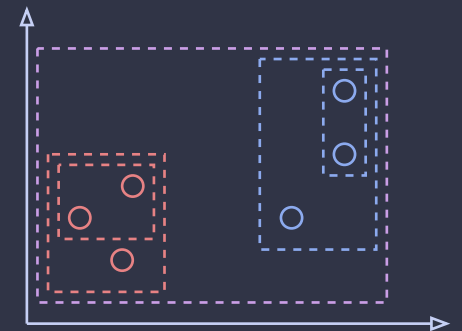
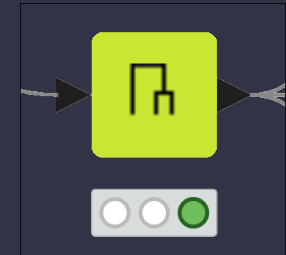
The *Hierarchical Clustering* node is provided by the standard Knime library.

The hierarchical clustering can be categorized as follows:

- **Hierarchical**: split the dataset into overlapping cluster organized as a tree.
- **Exclusive/Overlapping**: each node can be assigned to more than one cluster. However, the knime implementation of this algorithm extract only exclusive clusters.
- **Complete**: each element is assigned to a cluster.

The main **parameters** of the **hierarchical clustering** algorithm are the **number of clusters**, the **distance function** and the **linkage**.

Hierarchical Clustering



Hierarchical Clustering Node

The *Hierarchical Clustering* node is provided by the standard Knime library.

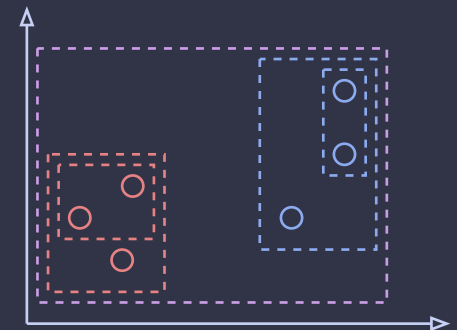
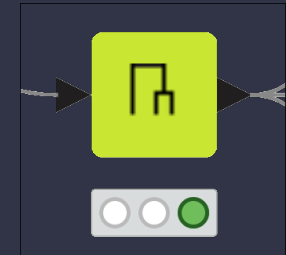
The hierarchical clustering can be categorized as follows:

- **Hierarchical**: split the dataset into overlapping cluster organized as a tree.
- **Exclusive/Overlapping**: each node can be assigned to more than one cluster. However, the knime implementation of this algorithm extract only exclusive clusters.
- **Complete**: each element is assigned to a cluster.

The main **parameters** of the **hierarchical clustering** algorithm are the **number of clusters**, the **distance function** and the **linkage**.

Also for the **hierarchical clustering** the normalization step is required.

Hierarchical Clustering



Hands on (15 minutes)

- Load and prepare the **Churn_yn.xls** dataset
- **Normalized** the data
- Configure **k-means** and **Hierarchical Clustering** to split the dataset in 3 clusters.
- Evaluate the distribution of the **churn** variable in the 3 clusters using a **Pie Chart**

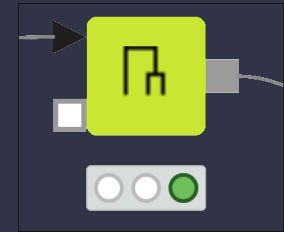


Proximity

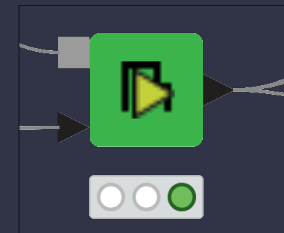
Hierarchical Clustering (DistMatrix)

The *Hierarchical Clustering (DistMatrix)* node provided by knime perform the clustering task starting from a distance matrix or a distance function.

Hierarchical Clustering (DistMatrix)



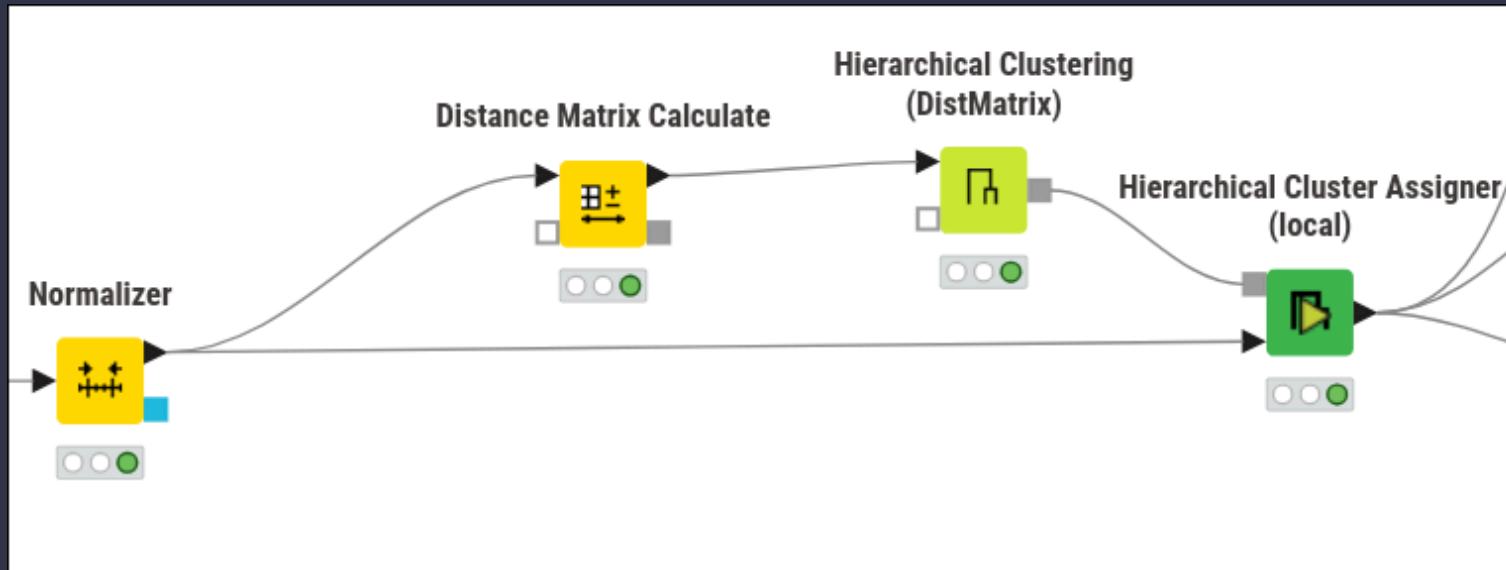
Hierarchical Cluster Assigner



Hierarchical Clustering (DistMatrix)

The *Hierarchical Clustering (DistMatrix)* node provided by knime perform the clustering task starting from a distance matrix or a distance function.

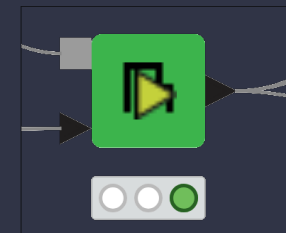
Distance Matrix



Hierarchical Clustering (DistMatrix)



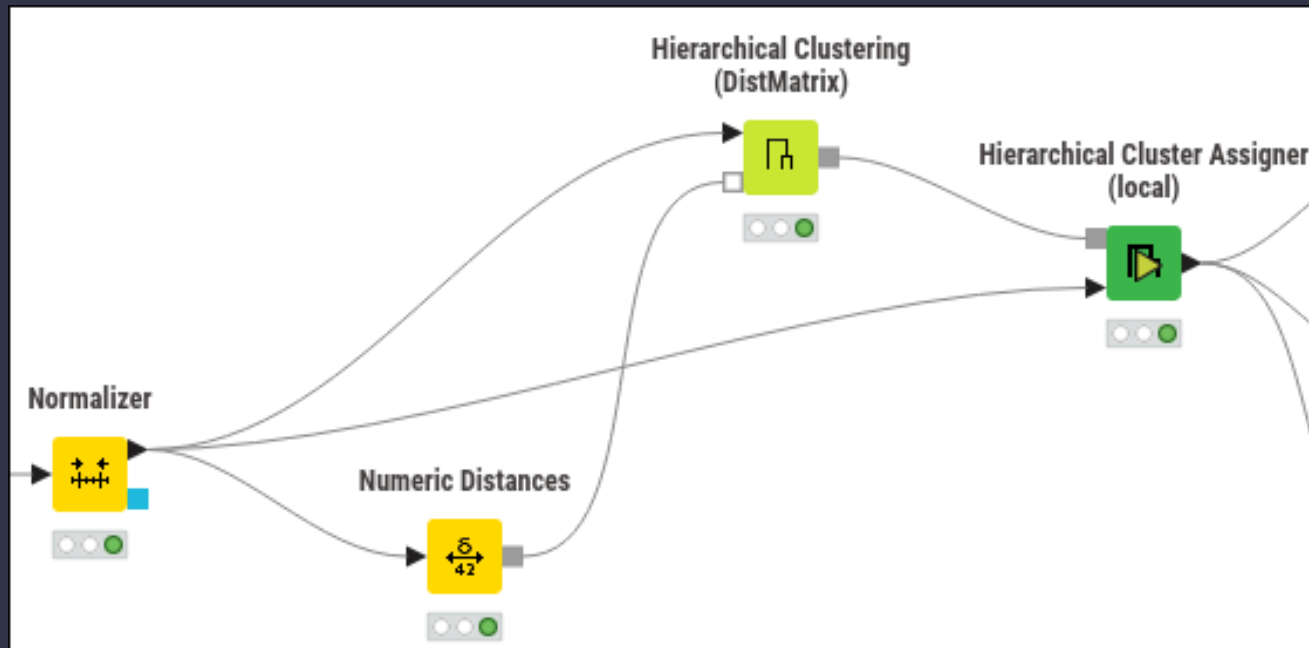
Hierarchical Cluster Assigner



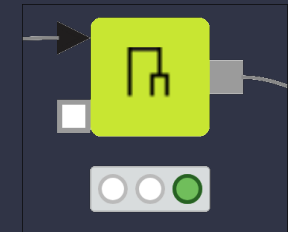
Hierarchical Clustering (DistMatrix)

The *Hierarchical Clustering (DistMatrix)* node provided by knime perform the clustering task starting from a distance matrix or a distance function.

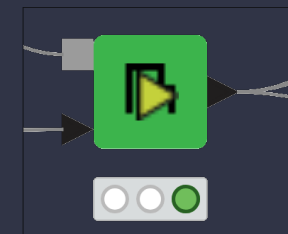
Distance function



Hierarchical Clustering (DistMatrix)



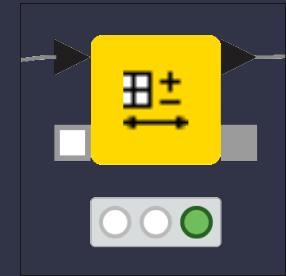
Hierarchical Cluster Assigner



Distance matrix Calculate

The *Distance Matrix Calculate* node provided by Knime calculates distance values for all pairs of rows of the input table.

Distance Matrix Calculate

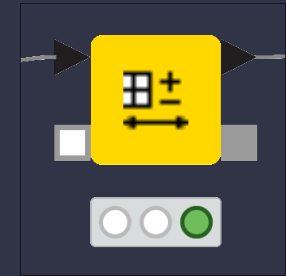


Distance matrix Calculate

The *Distance Matrix Calculate* node provided by Knime calculates distance values for all pairs of rows of the input table.

This node can be configured to compute many different distances:

Distance Matrix Calculate



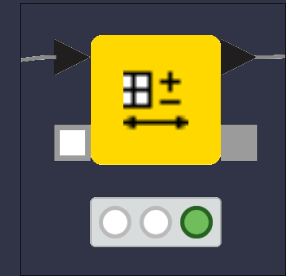
Distance matrix Calculate

The *Distance Matrix Calculate* node provided by Knime calculates distance values for all pairs of rows of the input table.

This node can be configured to compute many different distances:

The screenshot shows the configuration window for the 'Distance Matrix Calculate' node. It has four tabs: 'Distance Calculate' (selected), 'Flow Variables', 'Job Manager Selection', and 'Memory Policy'. The 'Distance Selection' dropdown is set to 'Euclidean'. A list of distance metrics is shown below it: Euclidean, Manhattan, Distance Vector, Tanimoto, OldTanimoto (deprecated), Cosine, Bitvector Cosine, and Dice. A red box highlights the 'Exclude' section, which includes a 'Filter' button and a radio button for 'Enforce exclusion' (which is selected). A green box highlights the 'Enforce inclusion' section, which includes a list of metrics (Eve Charge, Night Charge, Intl Charge) and a radio button for 'Enforce inclusion'. The 'Append Column Name' field is set to 'Distance'. The 'Chunk Size' is set to 1,000.

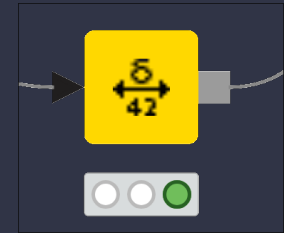
Distance Matrix Calculate



Numeric Distance

The *Numeric Distance* node provided by Knime implements the **Euclidean distance**, the **Manhattan distance** and the **Cosine Distance**

Numeric Distance

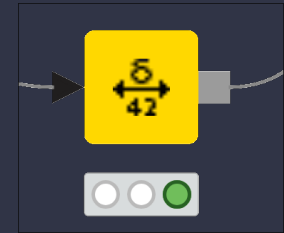


Numeric Distance

The *Numeric Distance* node provided by Knime implements the **Euclidean distance**, the **Manhattan distance** and the **Cosine Distance**

This node can be used in combination with the *Hierarchical Clustering* node.

Numeric Distance



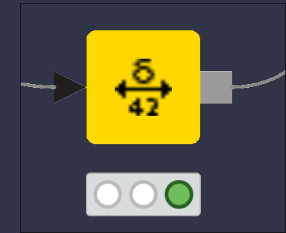
Numeric Distance

The *Numeric Distance* node provided by Knime implements the **Euclidean distance**, the **Manhattan distance** and the **Cosine Distance**

This node can be used in combination with the *Hierarchical Clustering* node.

The screenshot shows the 'Distance Configuration' tab of the Knime Numeric Distance node. It features two main sections: 'Exclude' (outlined in red) and 'Include' (outlined in green). The 'Exclude' section has a filter input and a list area that currently says 'No columns in this list'. The 'Include' section has a filter input and a list containing 'Day Mins', 'Eve Mins', 'Night Mins', 'Intl Mins', and 'CustServ Calls'. Between these sections are four arrow buttons: '>', '>>', '<', and '<<'. At the bottom, the 'Distance Selection' dropdown is set to 'Cosine Distance'. The 'Manual Selection' radio button is selected, and the 'Enforce exclusion' radio button is also selected.

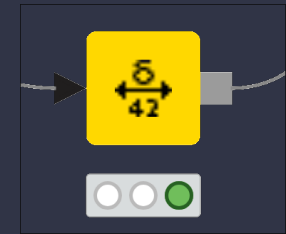
**Numeric
Distance**



Mahalanobis Distance

The *Mahalanobis Distance* node is provided by the Knime internal library.

**Numeric
Distance**

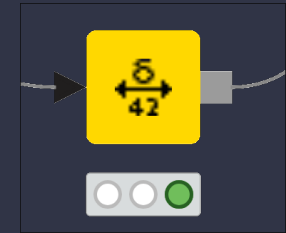


Mahalanobis Distance

The *Mahalanobis Distance* node is provided by the Knime internal library.

This node can be used in combination with the *Hierarchical Clustering* node.

**Numeric
Distance**

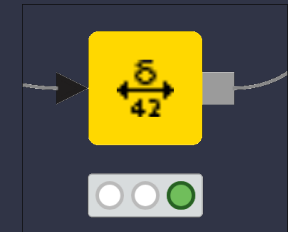


Mahalanobis Distance

The *Mahalanobis Distance* node is provided by the Knime internal library.
This node can be used in combination with the *Hierarchical Clustering* node.

The screenshot shows the configuration window for the Mahalanobis Distance node. It has four tabs: "Distance Configuration", "Flow Variables", "Job Manager Selection", and "Memory Policy". The "Distance Configuration" tab is active. At the top, there are two radio buttons: "Manual Selection" (selected) and "Wildcard/Regex Selection". Below these, there are two main sections: "Exclude" and "Include". The "Exclude" section is highlighted with a red border and contains a "Filter" input field with the text "No columns in this list" and a radio button labeled "Enforce exclusion". The "Include" section is highlighted with a green border and contains a "Filter" input field, a list of columns with checkboxes, and a radio button labeled "Enforce inclusion". The list of columns includes: "Day Mins", "Eve Mins", "Night Mins", "Intl Mins", "Day Charge", "Eve Charge", and "Night Charge". Between the two sections are four arrow buttons: ">", ">>", "<", and "<<".

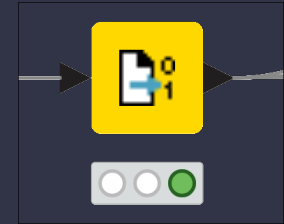
Numeric Distance



Jaccard Similarity

The *Create Bit Vector* node in combination with the *Bit Vector Distance* node, allows to implement the Jaccard similarity.

Create Bit Vector



**Bit Vector
Distance**



Jaccard Similarity

The *Create Bit Vector* node in combination with the *Bit Vector Distance* node, allows to implement the Jaccard similarity.

The *Create Bit Vector* allows to select a set of integer variables and combine them into a single bit-vector column:

Default Settings | Flow Variables | Job Manager Selection | Memory Policy

Bit vectors from multiple columns

☐ Create bit vectors from multiple string columns

Pattern: ☐ Contains wildcards ☐ Set bit if pattern ☒ does match ☐ does not match

☐ Case sensitive match ☐ Regular expression

☒ Create bit vector from multiple numeric columns

Threshold: ☐ Use percentage of the mean Percentage:

☒ Manual Selection ☐ Wildcard/Regex Selection

Exclude

No columns in this list

Include

☐ Int'l Plan ☐ VMail Plan

☒ Enforce exclusion ☐ Enforce inclusion

Bit vectors from a single column

☐ Create bit vectors from a single string column

Kind of string representation:

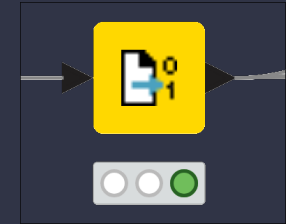
☐ Create bit vectors from a single collection column

Single column to be parsed:

General

☐ Remove column(s) used for bit vector creation Output column: ☐ Fail on invalid input Bit vector type: ☒ Dense ☐ Sparse

Create Bit Vector



Bit Vector
Distance

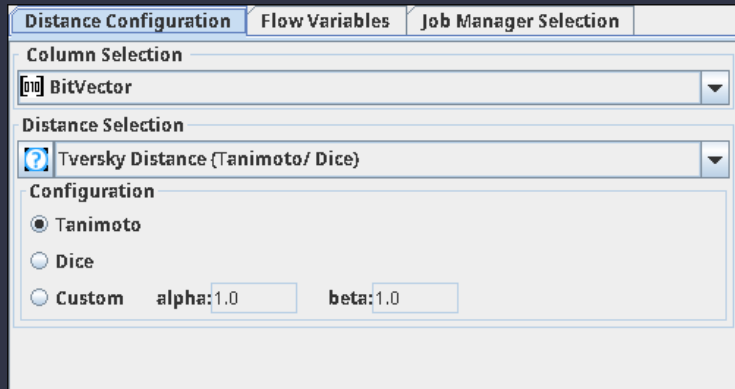


A	B	C	BitVec
0	0	0	000
0	1	0	010
1	1	1	111

Jaccard Similarity

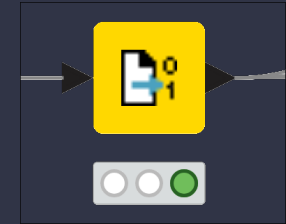
The *Create Bit Vector* node in combination with the *Bit Vector Distance* node, allows to implement the Jaccard similarity.

The *Bit Vector Distance* node allows to compute the many different distances between two **Bit Vectors**.



The **Tanimoto** distance, for **binary** features, is equivalent to the **Jaccard Similarity**.

Create Bit Vector



Bit Vector Distance



Hands on (20 minutes)

- Load the **Churn_yn.xlsx** dataset
- **Select** only the rows with **State=Ny** or **State=Wv**
- **Remove** the columns **State**, **Area Code** and **Phone**
- **Compute** the following distances:
 - Mahalanobis Distance on all the features **double** type
 - Euclidean Distance on all the features with **double** type
 - *Distance Matrix Calculate Node*
 - Cosine Distance on **Day Mins**, **Eve Mins**, **Night Mins**, **Intl Mins**, **CustServ Calls**
 - *Numeric Distances Node*
 - Jaccard Coefficient on **Int'l Plan** and **VMail Plan**.
 - *Create Bit Vector node + Bit Vector Distance*
- From those distances train: **Hierarchical Clustering (DistMatrix)**
- Use the **Hierarchical Cluster Assigner** node with **3 clusters** to predict the clusters.

